선생님 개인정보 · 회원가입 · 보안 가이드

작성일 2026-04-09 · 교사 대상 서비스의 개인정보 저장·암호화·해킹 대응 원리

> 핵심 원칙 한 문장: **"DB가 통째로 유출돼도 공격자가 로그인 못 하고 개인정보도 못 읽게"** 만드는 것이 목표다.

1. 교사 개인정보의 3가지 분류

모든 데이터는 아래 3가지로 분류하고 **분류별로 저장 방식을 다르게** 해야 한다.

분류	예시	저장 방식	비고
🔴 **절대 복호화 불가** (One-way)	비밀번호	bcrypt / argon2 해시	원본 복구 불가, 로그인 시 비교만
🟡 **복호화 가능** (Two-way 암호화)	주민번호, 전화번호, 이메일, 주소	AES-256-GCM 대칭 암호화	관리자·본인만 복호화 조회 가능
🟢 **평문 저장 OK**	닉네임, 학교명, 학년, 가입일	그대로 저장	공개돼도 피해 없음

> ❌ **절대 하면 안 되는 것:** 비밀번호를 평문 저장, MD5/SHA-256만 쓰기 (해시지만 무작위 대입에 취약), 주민번호를 그냥 varchar로 넣기, API 응답에 해시/암호문 노출

2. 비밀번호 — 왜 bcrypt인가?

2-1. 단순 해시의 문제점

```
SHA-256("password123") → "ef92b778..."  (항상 동일)
```

공격자가 레인보우 테이블(미리 계산된 해시 사전)로 1초 안에 원본 복원 가능.

2-2. bcrypt의 해결책

기법	설명
Salt	비밀번호마다 랜덤 문자열을 섞어서 해시 → 같은 비밀번호도 다른 결과
Work Factor	해시 계산을 일부러 느리게 (cost=10 → 약 100ms) → 공격자도 1초에 10번만 시도 가능
Adaptive	하드웨어 빨라지면 cost 올려서 시간 유지

2-3. 실제 구현 (Node.js)

```
import bcrypt from 'bcrypt';

// 회원가입 시
const hash = await bcrypt.hash(rawPassword, 10);
await db.users.insert({ loginId, passwordHash: hash });

// 로그인 시
const user = await db.users.findOne({ loginId });
const ok = await bcrypt.compare(rawPassword, user.passwordHash);
if (!ok) throw new Error('비밀번호 불일치');
```

> 💡 **핵심:** DB가 유출돼도 공격자는 해시만 보고 원본 비밀번호를 못 얻는다. bcrypt cost=10이면 8자 영문숫자 비밀번호 무작위 대입에 약 **100년** 걸린다.

3. 주민번호·전화번호 — AES-256-GCM 대칭 암호화

3-1. 대칭 암호화란?

열쇠(key)가 하나 있어서 **잠글 때도 열 때도 같은 열쇠**를 쓰는 방식. 비밀번호 해시와 달리 **복호화 가능**.

3-2. 저장 구조

```
users 테이블
  ├─ id (pk)
  ├─ loginId
  ├─ passwordHash       ← bcrypt
  ├─ nameEncrypted      ← AES 암호문
  ├─ phoneEncrypted     ← AES 암호문
  ├─ emailEncrypted     ← AES 암호문
  └─ ...

별도 안전한 곳 (Vercel 환경변수, AWS KMS 등)
  └─ ENCRYPTION_KEY = "a4b8...xyz (32 bytes)"
```

3-3. 왜 GCM 모드인가?

항목	설명
AES-256	군용급 암호 알고리즘, 현존 최강
GCM 모드	암호화 + 변조 감지(Authentication Tag)를 동시에 수행
IV(초기화벡터)	매번 랜덤 → 같은 평문도 다른 암호문
Auth Tag	공격자가 암호문 1글자만 바꿔도 복호화 시 에러

3-4. 실제 구현 (Node.js)

```
import crypto from 'crypto';

const KEY = Buffer.from(process.env.ENCRYPTION_KEY, 'hex'); // 32 bytes

export function encrypt(plaintext: string): string {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', KEY, iv);
  const encrypted = Buffer.concat([
    cipher.update(plaintext, 'utf8'),
    cipher.final(),
  ]);
  const authTag = cipher.getAuthTag();
  // iv(12) | tag(16) | ciphertext 를 이어붙여 base64로 저장
  return Buffer.concat([iv, authTag, encrypted]).toString('base64');
}

export function decrypt(payload: string): string {
  const buf = Buffer.from(payload, 'base64');
  const iv = buf.subarray(0, 12);
  const tag = buf.subarray(12, 28);
  const ct = buf.subarray(28);
  const decipher = crypto.createDecipheriv('aes-256-gcm', KEY, iv);
  decipher.setAuthTag(tag);
  return Buffer.concat([decipher.update(ct), decipher.final()]).toString('utf8');
}
```

> 💡 **검색 필요한 필드 팁:** 이메일처럼 "중복 체크"가 필요한 필드는 암호화된 값으로 검색이 불가능하다.
> → `emailHash` 컬럼을 하나 더 만들어 **SHA-256(이메일)**을 저장하고, 중복 체크는 이 해시로 한다. 암호문은 따로 저장.

4. 교사 회원가입 시스템 단계별 설계

4-1. 가입 플로우

```
1. 아이디 입력 → 중복 체크 (평문 가능)
2. 비밀번호 입력 → 강도 검사 (최소 10자, 영문+숫자+특수문자)
3. 이메일 입력 → 형식 검증 + emailHash 중복 체크
```

```
4. 이름/학교/전화 입력 → 형식 검증만
5. 약관 동의 (필수/선택 분리)
6. 제출
   ├─ 비밀번호: bcrypt 해시
   ├─ 이메일/이름/전화: AES-256 암호화
   └─ DB 저장 (status='PENDING' — 교사 인증 전까지 로그인 제한)
7. 교사 인증 (NEIS PDF 업로드) → status='ACTIVE'
```

4-2. 가입 시 저장되는 컬럼

컬럼	타입	저장 방식
id	bigint PK	자동생성
loginId	varchar(50)	평문, 유니크
passwordHash	varchar(60)	bcrypt
nameEncrypted	text	AES-256-GCM
emailEncrypted	text	AES-256-GCM
emailHash	varchar(64)	SHA-256 (중복 체크용)
phoneEncrypted	text	AES-256-GCM
schoolName	varchar(100)	평문 (공개 정보)
status	varchar(20)	PENDING / ACTIVE / SUSPENDED
createdDate	timestamp	자동

4-3. 약관·동의 원칙

- **필수:** 서비스 이용약관, 개인정보 수집·이용 동의
- **선택:** 마케팅 수신, 이벤트 알림
- **명시:** 수집 항목 / 이용 목적 / 보유 기간 / 파기 시점
- **저장:** 동의 시각, 동의 버전(약관 개정 시 재동의)

4-4. 교사 인증 단계 (이미 구현 완료)

- NEIS 급여명세서 PDF 업로드
- 이름·학교·재직상태·지급년월·푸터 파싱
- 푸터 ↔ 본문 교차검증으로 위변조 방지
- 최근 3개월 이내만 허용
- **민감 데이터(급여 금액)는 저장하지 않고 즉시 파기**
- 성공 시 status: PENDING → ACTIVE

5. 해킹 시나리오별 대응

공격	피해	방어
DB 덤프 유출	비밀번호 해시, 개인정보 암호문 노출	bcrypt cost=10 이상 + AES 키를 DB 바깥 저장
SQL Injection	임의 쿼리 실행	ORM(TypeORM) 또는 파라미터 바인딩만 사용
XSS (스크립트 삽입)	세션 토큰 탈취	React 기본 이스케이프 + HttpOnly 쿠키 세션
CSRF	본인 몰래 요청 전송	SameSite 쿠키 + CSRF 토큰
무작위 로그인 시도 (brute force)	비밀번호 추측	레이트 리밋 (IP당 분당 5회), 계정 잠금
환경변수 유출	AES 키 유출 → 모든 암호문 복호화	Vercel Secrets 사용, 키 로테이션 (6개월)
세션 하이재킹	타인 세션 도용	HTTPS 강제, 짧은 만료 + refresh token 분리

6. 개인정보보호법 준수 체크리스트

항목	내용
목적 명시	수집 항목과 이용 목적을 회원가입 시 명확히 고지
동의 수집	필수/선택 구분 + 동의 시각 저장
암호화 저장	주민번호·비밀번호·계좌번호는 암호화 의무 (개인정보보호법 제29조)
최소 수집	서비스에 꼭 필요한 정보만 — 주민번호는 수집 안 하는 것이 최선
보유 기간	목적 달성 시 즉시 파기, 최대 보유 기간 명시
파기 절차	탈퇴 시 30일 내 완전 삭제 (soft delete 아님)
접근 로그	관리자의 개인정보 조회는 로그 기록
개인정보처리방침	홈페이지에 공개 필수
위탁 고지	Vercel/Neon 등 외국 업체 사용 시 고지

> ⚠️ **온리티칭은 주민번호 수집 금지 권장.** NEIS 급여명세서로 신원이 이미 확인되므로 주민번호는 불필요. 수집 안 하면 개인정보보호법 제24조의2 적용 대상에서 벗어나 관리 부담이 크게 줄어든다.

7. 온리티칭 즉시 적용 TODO

우선순위	작업	효과
P0	users.password → bcrypt 해시로 마이그레이션	DB 유출 시 비밀번호 보호
P0	ENCRYPTION_KEY 환경변수 생성 + 암호화 유틸	개인정보 암호화 준비
P0	회원가입 시 이름/이메일/전화 AES 저장	개인정보 암호문화
🟡 P1	로그인 레이트 리밋 (IP당 분당 5회)	무작위 대입 차단
🟡 P1	HttpOnly + SameSite 쿠키 세션 전환	XSS/CSRF 완화
🟢 P2	관리자 개인정보 조회 감사 로그	내부자 위협 대비
🟢 P2	개인정보처리방침 페이지 작성	법적 의무

한 줄 요약

> **비밀번호는 bcrypt로 복구 불가능하게, 주민번호·전화는 AES-256-GCM으로 복호화 가능하게, 키는 DB 바깥에, 교사 인증은 NEIS PDF로, 최소 수집 원칙 지키면 끝.**